

PROGRAMMING FOR PROBLEM SOLVING

UNIT 1 C Fundamentals & Decision Making

PREMANAND S

Assistant Professor

Department of Computational Intelligence

SRM Institute of Science and Technology

Kattankulathur Campus

First Year Engineering

One Condition...
Two Possibilities!

if - else in C

When the condition is TRUE → do one thing
When it is FALSE → do another thing!

Every decision has two sides!

1 What is if - else ?

The if-else statement allows the program to execute one set of statements when the condition is TRUE and another set when the condition is FALSE.

It helps the program to make a decision!

2 Syntax

```
if (condition)
{
    // statements when
    // condition is TRUE
}
else
{
    // statements when
    // condition is FALSE
}
```

3 Flowchart

```

graph TD
    Start([Start]) --> Condition{Condition ?}
    Condition -- TRUE --> ExecuteTrue[Execute statements in if block]
    Condition -- FALSE --> ExecuteFalse[Execute statements in else block]
    ExecuteTrue --> End([End])
    ExecuteFalse --> End
    
```

4 Example 1 Pass or Fail

```
#include <stdio.h>
int main()
{
    int marks;
    printf("Enter marks: ");
    scanf("%d", &marks);
    if (marks >= 40)
        printf("Pass");
    else
        printf("Fail");
    return 0;
}
```

Marks >= 40 ?
Pass
otherwise
Fail!

5 Example 2 Even or Odd

```
#include <stdio.h>
int main()
{
    int number;
    printf("Enter a number: ");
    scanf("%d", &number);
    if (number % 2 == 0)
        printf("Even");
    else
        printf("Odd");
    return 0;
}
```

Remainder 0 ?
Even
else
Odd!

2 4 6 8 ...

Even

1 3 5 7 ...

Odd

6 Example 3 Positive or Negative

```
#include <stdio.h>
int main()
{
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n >= 0)
        printf("Positive");
    else
        printf("Negative");
    return 0;
}
```

Is the number >= 0 ?

Positive

Negative

7 Example 4 Using Logical Operator

```
#include <stdio.h>
int main()
{
    int age, hasID;
    printf("Enter your age: ");
    scanf("%d", &age);
    printf("Do you have ID?");
    scanf("%d", &hasID);
    if (age >= 18 && hasID == 1)
        printf("Allowed");
    else
        printf("Not Allowed");
    return 0;
}
```

Both conditions must be TRUE!

8 if vs if - else

if

- Executes statements only when the condition is TRUE.
- If the condition is FALSE, statements are skipped.

if - else

- Executes one set of statements when TRUE and another set when FALSE.
- More control over decisions.

9 Practice Questions

- Write a C program to check whether a number is greater than 100 or not.
- Write a C program to check whether a number is divisible by 5 or not.
- Write a C program to check whether a person is eligible to vote (age >= 18) or not.
- Write a C program to check whether a number is even or odd.
- Write a C program to check whether a student has passed or failed (marks >= 40).

10 Key Takeaways

- ✓ if - else handles two possible outcomes.
- ✓ Use relational and logical operators in conditions.
- ✓ Write clear and meaningful messages for both cases.
- ✓ Always test with different inputs.

Code. Think. Decide. Learn!

Small conditions make big decisions in real life... and in programming!

Keep Practicing!

Now we handle BOTH cases!



if – else in C

When the condition is **TRUE** → do one thing
When it is **FALSE** → do another thing!

Every condition has two paths!



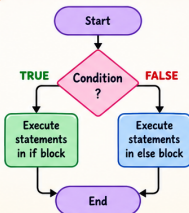
1 Syntax

```
if (condition)
{
    // statements when
    // condition is TRUE
}
else
{
    // statements when
    // condition is FALSE
}
```



The else part is executed only when the condition is FALSE.

2 Flowchart



3 Example Program (Positive or Negative)

```
#include <stdio.h>
int main()
{
    int number;
    printf("Enter a number: ");
    scanf("%d", &number);

    if (number >= 0)
    {
        printf("Positive");
    }
    else
    {
        printf("Negative");
    }

    return 0;
}
```

4 Sample Output

Input: 25

Enter a number: 25
Positive

Condition is TRUE
→ if block executes



Input: -10

Enter a number: -10
Negative

Condition is FALSE
→ else block executes



5 Real-life Analogy



If it is raining → take an umbrella
else → don't take an umbrella.

6 Key Points

- ✓ if-else handles two possible outcomes.
- ✓ The condition is evaluated only once.
- ✓ Use meaningful messages for both cases.
- ✓ We can use relational and logical operators in the condition.

7 Next Steps



Write better logic. Build smarter programs.



Practice. Improve. Excel!



Sample Program — if--else

C Program

```
#include <stdio.h>
int main()
{
    int marks;
    printf("Enter your marks: ");
    scanf("%d", &marks);
    if (marks >= 40)
    {
        printf("Pass");
    }
    else
    {
        printf("Fail");
    }
    return 0;
}
```

Problem 1

Problem

A student is allowed to enter a special laboratory only if:

- Age is at least 18.
- The student has a valid ID.
- The student has registered for the laboratory.

Display **“Entry Allowed”** if all conditions are satisfied; otherwise display **“Entry Denied”**.

Problem 2

Problem

A student receives a scholarship if:

- Marks are at least 85.
- Attendance is at least 75%.
- The student has no disciplinary issue.

Display **“Scholarship Approved”** if all conditions are satisfied; otherwise display **“Scholarship Not Approved”**.

Problem 3

Problem

Write a C program to check whether a number is **between 10 and 100**. Display **“Valid”** if both conditions are satisfied; otherwise display **“Invalid”**.

Problem 4

Problem

Write a C program to check whether a person can participate in a competition if:

- Age is between 18 and 30.
- The person has registered.

Display **“Eligible”** if both conditions are satisfied; otherwise display **“Not Eligible”**.

Problem 5

Problem

Write a C program to check whether a user is allowed to log in. The login is successful only if the entered username is correct **and** the password is correct. Display **“Login Successful”** or **“Invalid Login”**.

Problem 6

Problem

Write a C program to check whether a student is **eligible for the examination**.

The student is eligible only if:

- Attendance is at least 75%, and
- Examination fee is paid.

Otherwise, display **“Not Eligible”**.

Problem 7

Problem

Write a C program to check whether a person can participate in a competition if:

- The age is between 18 and 30, and
- The person has registered.

Display **“Eligible”** if both conditions are satisfied; otherwise display **“Not Eligible”**.

Problem 8

Problem

Write a C program to check whether a given number is a **Special Number**.

A number is special if:

- The number is between 1 and 100, **AND**
- The number is either even **OR** divisible by 5.

Display **“Special Number”** if the conditions are satisfied; otherwise display **“Not a Special Number”**.

Some Concepts to be understand!

1 if vs if-else

if (only)

- Executes the block only when the condition is TRUE.
- If the condition is FALSE, nothing happens.

```
#include <stdio.h>
int main()
{
    int marks = 30;
    if (marks >= 40)
        printf("Pass");
    return 0;
}
```

Output: (nothing is printed because condition is FALSE)

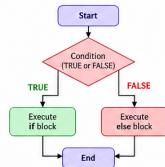
if-else

- Executes one block when the condition is TRUE.
- Executes the other block when the condition is FALSE.

```
#include <stdio.h>
int main()
{
    int marks = 30;
    if (marks >= 40)
        printf("Pass");
    else
        printf("Fail");
    return 0;
}
```

Output: Fail

2 How if-else Works



- The condition is evaluated.
- If it is TRUE, the if block executes.
- If it is FALSE, the else block executes.
- Exactly one of the two blocks is executed.

3 Condition Produces TRUE or FALSE

- The expression inside if (...) must produce a TRUE or FALSE value.
- We can use relational, logical, or other conditions.

```
#include <stdio.h>
int main()
{
    int age = 20;
    if (age >= 18)
        printf("Adult");
    else
        printf("Minor");
    return 0;
}
```

Some examples of conditions:

- age >= 18
- number == 0
- number % 2 == 0
- (age >= 18 && hasID == 1)

4 0 and Non-zero Values in C

- In C, the value 0 is considered FALSE.
- Any non-zero value is considered TRUE.

```
#include <stdio.h>
int main()
{
    int x = 10;
    if (x)
        printf("True");
    else
        printf("False");
    return 0;
}
```

Examples:

- if (0) → FALSE
- if (1) → TRUE
- if (-5) → TRUE
- if (100) → TRUE

5 = vs == (Very Important)

Operator	Meaning	Example
=	Assignment	marks = 40;
==	Comparison	marks == 40

- Using = inside a condition is a common mistake.
- We must use == to compare values.

Wrong:

```
if (marks = 40)
    // This assigns 40 to marks
```

Correct:

```
if (marks == 40)
    // This checks if marks is 40
```

6 Braces {} and Multiple Statements

- If there is only one statement, braces are optional (but still good practice to use them).
- When there are multiple statements, braces are required.

```
#include <stdio.h>
int main()
{
    int marks = 75;
    if (marks >= 40)
    {
        printf("Pass\n");
        printf("Congratulations!\n");
    }
    else
    {
        printf("Fail\n");
        printf("Try again!\n");
    }
    return 0;
}
```

7 else Belongs to the Nearest if

- When if statements are nested, the else is associated with the nearest unmatched if.

```
#include <stdio.h>
int main()
{
    int age = 20, hasID = 0;
    if (age >= 18)
        if (hasID == 1)
            printf("Allowed");
        else
            printf("Not Allowed");
    return 0;
}
```

The else in this example belongs to if (hasID == 1), not to the outer if.

8 Nested if (Preview)

- An if inside another if is called a nested if.

```
#include <stdio.h>
int main()
{
    int age = 20, hasID = 1;
    if (age >= 18)
    {
        if (hasID == 1)
            printf("Allowed");
    }
    return 0;
}
```

Detailed nested if examples will be covered in the next topic.

9 Common Mistakes

- ✗ if (marks = 40) // use ==
- ✗ if marks >= 40 // missing {}
- ✗ if (age >= 18); // extra ;
- ✗ else without a matching if
- ✗ not using braces for multiple statements

10 Predict the Output

- Try to find the output without running the program.

```
#include <stdio.h>
int main()
{
    int x = 15;
    if (x > 20)
        printf("A");
    else
        printf("B");
    return 0;
}
```

Output:
B



Why do we need **else if** ?

Recap: if - else

The if - else statement is used when there are only **TWO possible outcomes** (TRUE or FALSE).

```
if (condition)
{
    // do this
}
else
{
    // do this
}
```

A Real-World Example

Suppose we want to **assign a grade** to a student based on their marks.

Marks Range	Grade
90 - 100	A
75 - 89	B
60 - 74	C
40 - 59	D
Below 40	Fail

What if there are **more than two** possible outcomes?



Here, we have **FIVE different outcomes!**

Can we solve this using only if - else ?



No!

We need **else if** for multiple conditions.



Key Takeaway

We use **else if** when we need to check **multiple conditions** and handle **three or more possible outcomes**.

One condition.
Many possibilities!

Else IF - How it works?

else if Syntax in C

Used when there are **three or more** possible outcomes!

Same idea...
More choices!



General Syntax

```
if (condition1)
{
    statement1;
}
else if (condition2)
{
    statement2;
}
else if (condition3)
{
    statement3;
}
...
else
{
    statementN;
}
```

Explanation

- 1 The **first condition** is checked.
- 2 If it is FALSE, the **next else if** condition is checked.
- 3 This continues until a condition is **TRUE**.
- 4 When a condition is TRUE, the corresponding block executes, and the **rest are skipped**.
- 5 If none of the conditions are TRUE, the **else** block executes.

Example Structure

```
int marks = 75;
if (marks >= 90)
{
    printf("Grade A\n");
}
else if (marks >= 75)
{
    printf("Grade B\n");
}
else if (marks >= 60)
{
    printf("Grade C\n");
}
else if (marks >= 40)
{
    printf("Grade D\n");
}
else
{
    printf("Fail\n");
}
```

Key Points

- Use **else if** when there are **three or more conditions**.
- The conditions are checked **in order** (from top to bottom).
- Only the **first TRUE** condition is executed.
- The **else** part is optional, but useful to handle all other cases.

Write conditions from **specific** (highest) to **general** (lowest)!



Key Takeaway

else if allows us to handle **multiple conditions** in a decision-making sequence.

*Better Logic
Brighter Future!*

Else IF - Understanding



Example 1: Grade Calculation using else if

Write a C program to read a student's marks and display the grade using **else if**.

1 Problem

Read the marks obtained by a student (0 – 100) and display the grade as per the following criteria:

Marks Range	Grade
90 – 100	A
75 – 89	B
60 – 74	C
40 – 59	D
Below 40	Fail

2 C Program

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int marks;
6
7     printf("Enter the marks: ");
8     scanf("%d", &marks);
9
10    if (marks >= 90)
11        printf("Grade A");
12    else if (marks >= 75)
13        printf("Grade B");
14    else if (marks >= 60)
15        printf("Grade C");
16    else if (marks >= 40)
17        printf("Grade D");
18    else
19        printf("Fail");
20
21    return 0;
22 }
```

3 Sample Runs

Example 1:
High Score

Enter the marks: 95
Grade A

Excellent!

Example 2:
Middle Score

Enter the marks: 68
Grade C

Good Progress!

Example 3:
Low Score

Enter the marks: 32
Fail

Keep Trying!

4 Key Takeaways

- We use **else if** when there are multiple possible outcomes.
- Conditions are checked from top to bottom.
- Only the **first TRUE** condition is executed.
- The **else** block executes if none of the conditions are TRUE.



Try It Yourself!

Modify the program to display "Invalid Marks" if the input is less than 0 or greater than 100.

Same logic.
Different possibilities!



Practice Questions — else if

1 Number Classification

Write a C program to read a number and display whether it is **positive, negative, or zero** using if--else if--else.

2 Largest of Two Numbers

Write a C program to read two numbers and display which number is greater. If both numbers are equal, display **“Both are Equal”**.

3 Grade Calculation

Write a C program to read marks and display the grade:

- 90–100 → A
- 75–89 → B
- 60–74 → C
- 40–59 → D
- Below 40 → Fail

Use else if.

① Age Category

Write a C program to classify a person's age as:

- Below 13 → Child
- 13–19 → Teenager
- 20–59 → Adult
- 60 and above → Senior Citizen

② Temperature Classification

Write a C program to classify the temperature:

- Below 20 → Cold
- 20–29 → Moderate
- 30–39 → Hot
- 40 and above → Very Hot

Practice Questions — else if + Operators

1 Number Type

Write a C program to classify a number as:

- Positive and even
- Positive and odd
- Negative and even
- Negative and odd
- Zero

Use appropriate relational, modulus, and logical operators.

2 Electricity Bill Slab

Write a C program to calculate the electricity charge based on the units consumed:

- 0–100 units → Rs. 2 per unit
- 101–200 units → Rs. 3 per unit
- 201–300 units → Rs. 5 per unit
- Above 300 units → Rs. 7 per unit

Use else if to identify the slab.

1 Scholarship Eligibility

Write a C program to classify a student as:

- **Full Scholarship:** marks ≥ 90 AND attendance ≥ 85
- **Partial Scholarship:** marks ≥ 75 AND attendance ≥ 75
- **No Scholarship:** otherwise

Use `&&` appropriately.

2 Day Type

Write a C program to read a number representing a day (1–7) and display:

- 1–5 $\rightarrow$ Weekday
- 6–7 $\rightarrow$ Weekend
- Any other value $\rightarrow$ Invalid Day

Use `else if`.

1 Student Performance

Write a C program to classify a student's performance using marks and attendance:

- Marks ≥ 85 AND attendance $\geq 90 \rightarrow$ Excellent
- Marks ≥ 70 AND attendance $\geq 75 \rightarrow$ Good
- Marks ≥ 40 AND attendance $\geq 75 \rightarrow$ Satisfactory
- Otherwise $\rightarrow$ Needs Improvement

Use else if with logical operators.